

Space Invaders Design Document

Jevin Pauly

Table of Contents

Proxy Pattern	2
Factory Pattern	4
Iterator Pattern	6
Object Pooling	8
Command Pattern	10
State Pattern	12
Composite Pattern	14
Strategy Pattern	16
Visitor Pattern	18
Observer Pattern	20

Link to video explanation

<https://www.youtube.com/watch?v=RPqyw4HyHOE>

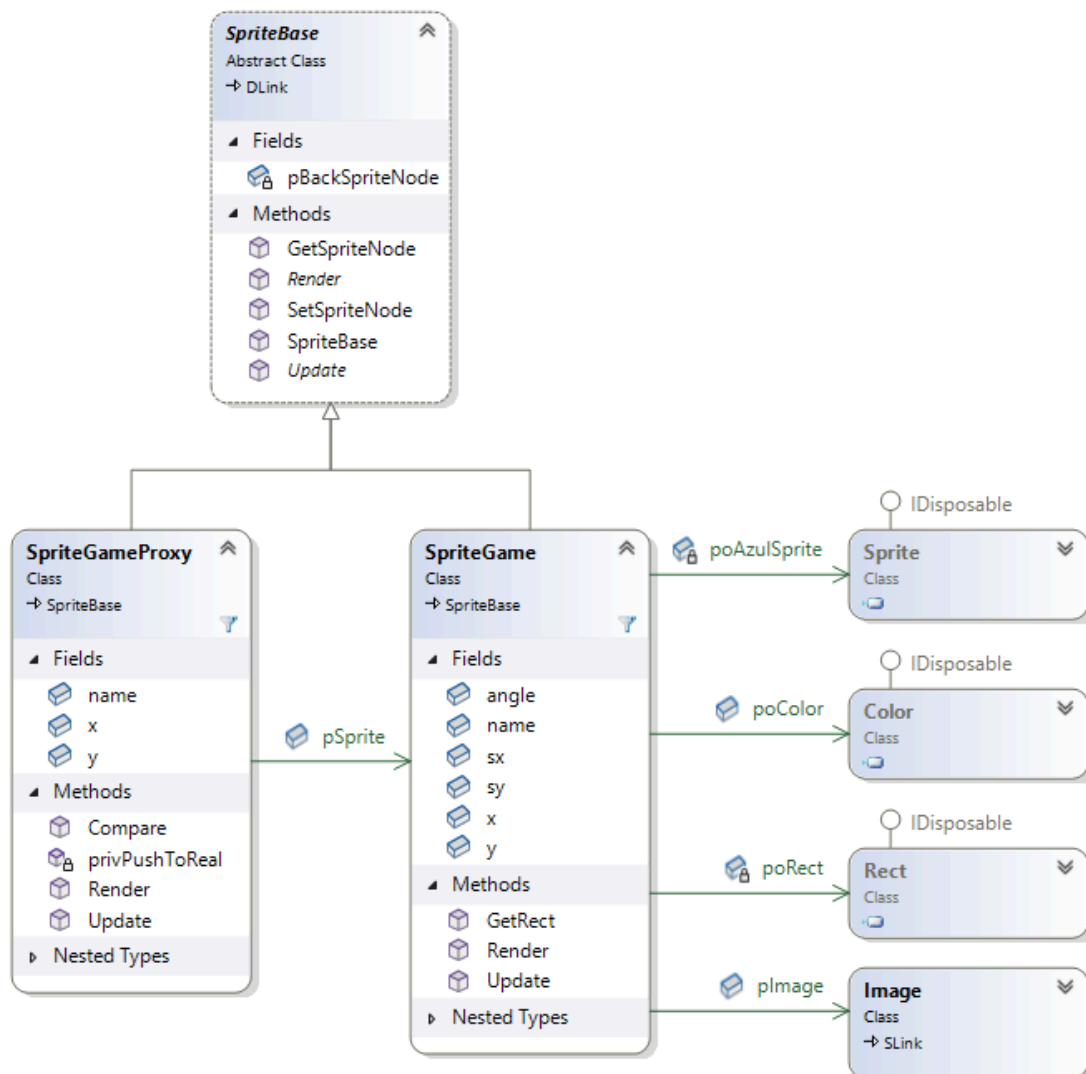
Proxy Pattern

Problem:

The alien grid in Space Invaders consists of 3 alien types: squids, octopi, and crabs. The total number of aliens in the grid is 55; creating game sprites for each alien is inefficient regarding memory usage and performance. We need to create a system to reuse the same 3 sprites to fill up the 55 aliens in the grid while also maintaining differences in their positions on the screen.

Solution:

We can go about reducing the number of heavy data objects, like game sprites, via the use of proxies that are more simple and lightweight. The proxy pattern allows us to create multiple sprite proxies that can all point to a real game sprite. These proxies can store unique x and y positions that we can then push onto the real game sprite. We will use them as a mediator that can deal with all the functionalities by passing them to the real game sprite.



Pattern Description:

By acting as a bridge between a client and an actual object, the proxy pattern gives access to the real object more control and a degree of indirection. This can serve multiple purposes, like restricting access privileges, postponing the creation or loading of the actual object until required, or offering extra functionality in relation to the functions of the actual object.

The proxy objects push their data, like coordinates, to the matching real game sprite objects when it's time to render the game sprites. This keeps the system design light and effective while guaranteeing that the sprites are shown on the screen as intended.

Key Object-Oriented Mechanics:

In terms of object-oriented mechanics, the proxy pattern leverages key principles such as abstraction and encapsulation. Abstraction is employed here through the definition of an interface shared by both the proxy and the real object, ensuring that we can interact with them using a unified set of methods (this is done through the use of a common parent class, Sprite Base). Encapsulation plays a crucial role in concealing the complexities of the real object behind the proxy, enabling the proxy to manage interactions with the real object internally.

Usage in Space Invaders:

Lightweight proxy objects are used to interact with game sprite objects instead of the heavy-weight ones directly. These proxies store important information, such as the location of specific aliens, and when needed, they transfer requests to the actual game sprite objects. This makes it possible to handle game sprite operations in a more optimized and regulated manner.

Each proxy holds the location of one alien (x, y). To display these aliens in the correct location on our screen, we use the proxies to push the coordinates onto the real game sprite and then render them. We only store and push important data to keep the proxies light in the data. Since multiple proxies can point to the same real game sprite, we only keep one of each alien (squid, octopus, and crab) in the game sprites. A similar implementation is done to render multiple sets of shield bricks that the ship can use for cover. Something to note here is that the sprites need to have the coordinates of the proxy pushed to them before it is rendered to get the correct placement of the game sprites.

Factory Pattern

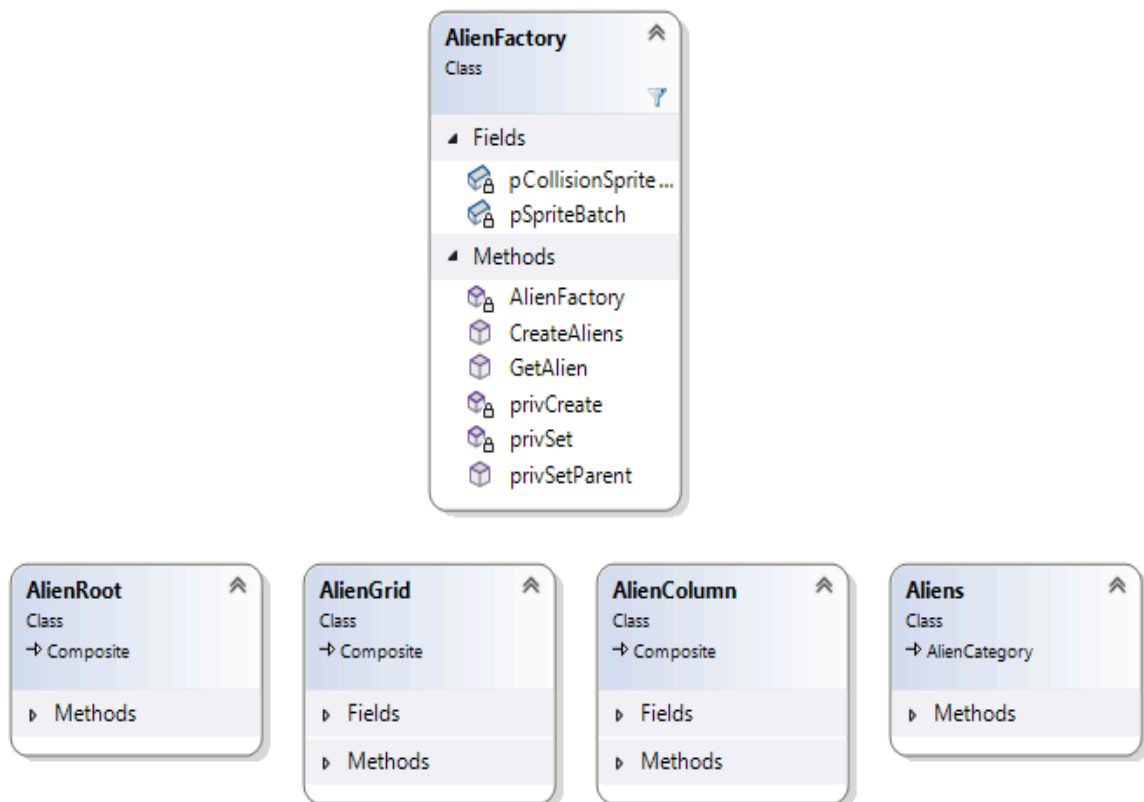
Problem:

The alien grid needs to be populated with alien game sprites. Manual creation of each sprite is a tedious and not user-friendly process. We need to come up with a solution to be able to reuse object creation/instantiation for objects that are created in mass, like shields and aliens.

Another problem is that we need to create different types of aliens and shield bricks according to the location.

Solution:

We can solve this issue by creating an interface that can be used to manage the creation of these objects. The Factory Pattern provides a centralized place for creating instances based on certain criteria or conditions. Additionally, factories can be reused across different parts of the application or even in different projects, providing a standardized approach to object creation.



Pattern Description:

The Factory Pattern is a design pattern that provides an interface for creating objects in a superclass but allows subclasses to alter the type of objects that will be created. We interact with the factory interface or method to request objects without needing to know the specific implementations or classes involved. This pattern allows for better flexibility and scalability in the object creation process.

The Factory Pattern can be considered a blueprint for object creation and instantiation in a standardized form, while also allowing variations in the types of objects based on subclass overriding and other conditions.

Key Object-Oriented Mechanics:

Regarding object-oriented mechanics, the Factory Pattern uses encapsulation by hiding the details of the object instantiation from the user. It also promotes modularity and reduces the coupling in the object creation code. Due to the nature of the pattern, users are dependent on the factory interface rather than directly instantiating concrete classes, which helps with flexibility and easier maintenance. This pattern uses polymorphism and inheritance to define the interface; subclasses are then used to implement the factory methods to return the required objects. This also helps improve readability for users and avoids code clutter via delegation.

Another important thing to note is that a factory is only responsible for a certain type of object; different types of objects require different factories for creation to maintain a standard (aliens have an alien factory, shields have a shield factory). As long as different object types have different factories for instantiation, we can minimize dependency issues and complexity.

Usage in Space Invaders:

The alien factory is used to create or reuse (depending on the scenario) aliens according to their specific location. It creates the appropriate alien based on the alien category type and its coordinates on the x and y axes. The factory is also used to create the columns and grid for the aliens. It is also responsible for linking the alien object with a game sprite and a collision box.

We also use it to assign the correct parent upon creation (grid as the parent of columns, column as the parent of aliens, etc.).

A similar factory is used for the creation of the shield bricks that sit between the ship and the aliens. When the factory is called to create the objects, it first checks to see if there is a similar object that it can reuse rather than attempting to create a new one first.

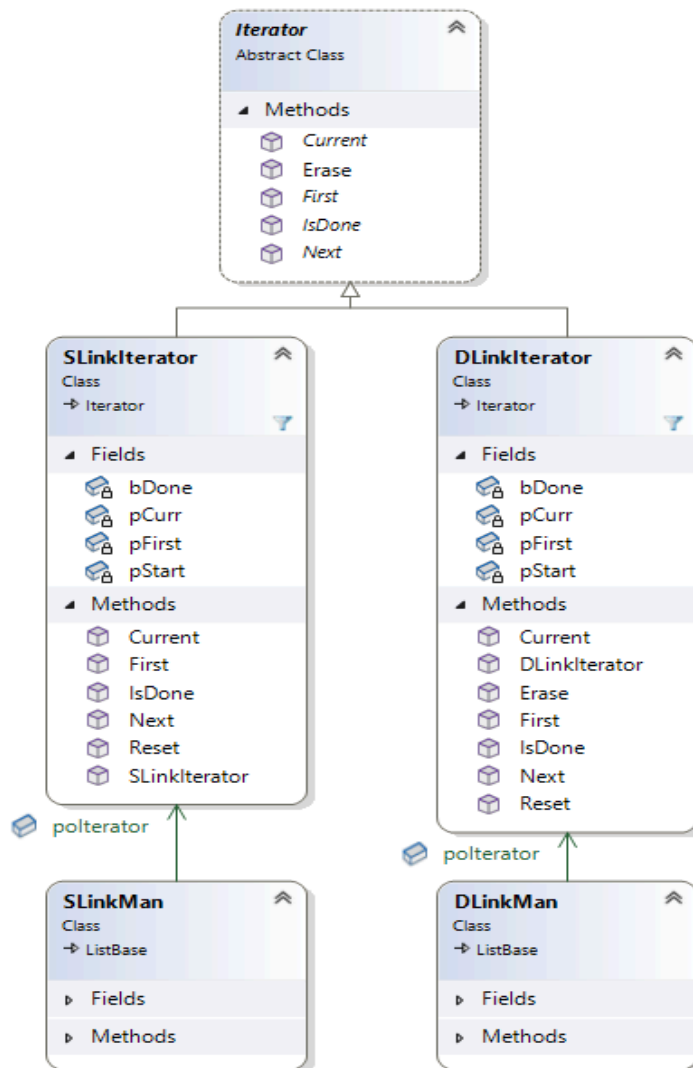
Iterator Pattern

Problem:

With several classes and objects involved, each using a single or double-linked list, we need to find a standardized way to traverse through the elements of such lists regardless of their internal structure and implementation.

Solution:

We can solve this issue with the help of the iterator pattern. An iterator pattern is used to access the elements of an aggregate, regardless of their structure. It also delegated the traversal duty to the iterator object. Since it provides a uniform standard for accessing elements in a collection, this helps in ensuring consistent access to the data without revealing the internal workings to the client.



Pattern Description:

The iterator pattern is used to act as an interface between the client and the actual traversal of the data structure. It includes methods that the client can call, such as Next(), First(), Curr(), and IsDone(). These methods are used to call the next value in the list, fetch the head of the list, grab the current node in the traversal, and check whether we are at the end of the list.

The iterator is created according to the collection type (aggregate); the iterator may include further helper functions such as Erase() to remove a node during traversal. By separating the concerns of iteration and aggregation, these patterns help create a clean and modular design.

Key Object-Oriented Mechanics:

The key object-oriented mechanics that the iterator pattern uses are abstraction and encapsulation. The pattern abstracts the process of iteration away from the underlying collection. Defining an iterator interface or abstract class separates the iteration logic from the details of the collection's implementation.

The iterator pattern encapsulates the iteration logic within the iterator object. The iterator maintains its state and controls the traversal of elements within the collection. By encapsulating this behavior, the pattern hides the complexity of iteration from the main code. We interact with the iterator to traverse through the underlying collection.

Usage in Space Invaders:

Space Invaders utilizes the Iterator Design Pattern to tackle the challenge of its diverse data structures. This pattern allows for a unified approach to iterating through the game's data, regardless of the underlying structure. By inheriting from a base Iterator class, the specific behavior is encapsulated, enabling users to navigate various data structures without delving into the complexities of their traversal mechanisms.

Most of the objects or items are controlled by their respective managers, some of which are Single Linked Lists whereas others are Double Linked Lists, lists. This is where iterators come in handy. We also use composite patterns to manage entities like aliens and shields; to traverse such trees, we use composite forward and reverse iterators. All the iterators, regardless of the types, follow similar functions, even though the iterator base class is different for the composite variants.

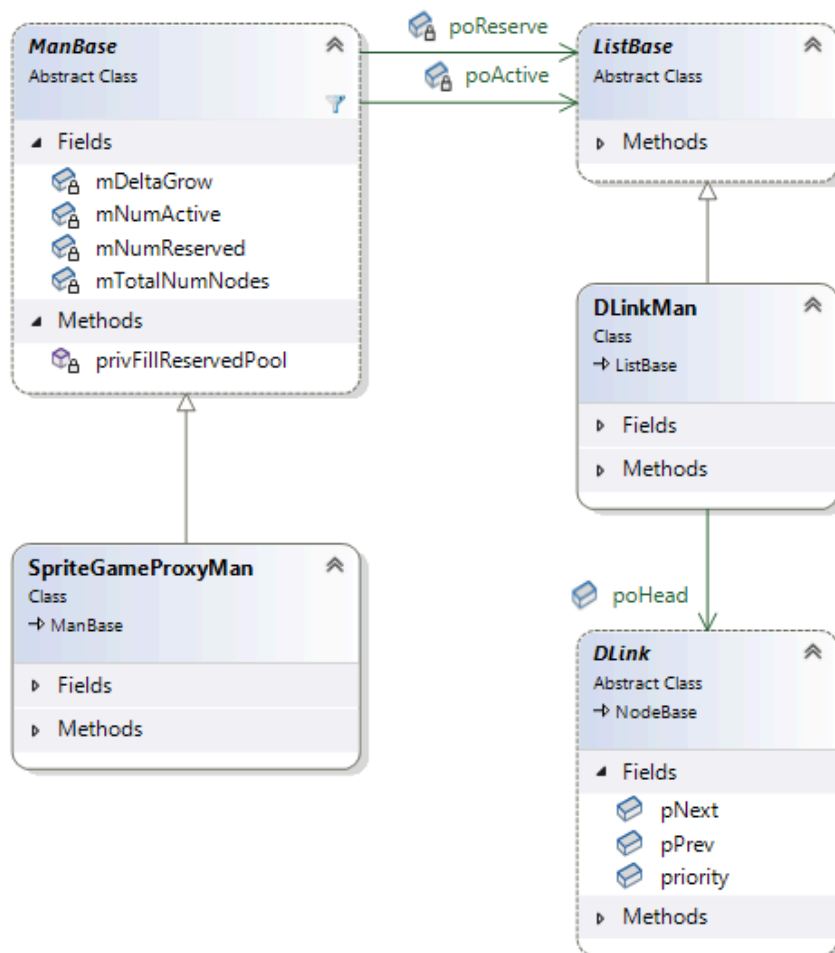
Object Pooling

Problem:

Creating and destroying objects (allocation and deallocation) can be slow for games, especially if it happens at the wrong time. Since we do not have a direct say on when the memory is freed, we need to find a way to bypass garbage collection or prevent it from occurring at the wrong time. Ideally, we want to avoid this deallocation entirely. But normally, unused objects get cleaned up automatically when nothing else references them (reference count hits zero).

Solution:

By building a “pool” of pre-made objects, object pooling addresses poor memory allocation and deallocation issues. Picture a bucket filled with aliens or shields. The game takes one from the pool when it needs a new one rather than making one from the start. Compared to continuously producing and deleting objects, this is much faster. The object is returned to the pool rather than disposed of when it is no longer needed. By doing this, the delay brought on by the garbage collector cleaning up at random times is avoided.



Pattern Description:

By reusing a group of objects rather than constantly producing and deleting them, object pooling is a software design technique that seeks to enhance performance and resource management. As part of the pattern, a pool of initialized objects is created and maintained, ready for usage. Whenever an object is required, it is pulled from the pool rather than being instantiated. Rather than being disposed of, the item is put back into the pool after usage. Especially in situations where object creation is costly or frequent, this method improves efficiency by lowering the overhead related to object creation, initialization, and deletion.

Key Object-Oriented Mechanics:

Object pooling is advantageous in applications, especially in languages where developers lack control over garbage collection timing. Its primary benefit lies in minimizing overhead caused by frequent dynamic allocation and deallocation of objects. Additionally, it fosters scalability and efficient resource management by restricting the creation of objects at runtime to essential instances. This approach simplifies resource handling, making it easier to manage application resources effectively. However, one significant trade-off of object pooling is that developers must handle it responsibly. When the reserve pool is not managed properly, more objects than necessary are created for the program. Setting suitable initial pool sizes and a reasonable growth size for reserve pools are essential.

Usage in Space Invaders:

In Space Invaders, things like aliens, UFOs, and your ship are constantly being removed or "killed." To keep track of them all, we created managers that act like buckets. These buckets hold two sets of objects: active ones currently in use and reserve ones ready to be used. By using object pooling with these managers, we avoided deleting and recreating dozens of objects all the time.

Every single manager class that is derived from the manager base class contains reserve and active pools with objects. We control the delta for reserve pool object creation and also how many objects need to be created before actual game execution. We need to be careful not to create a pool size that is excessively large to avoid unnecessary memory usage.

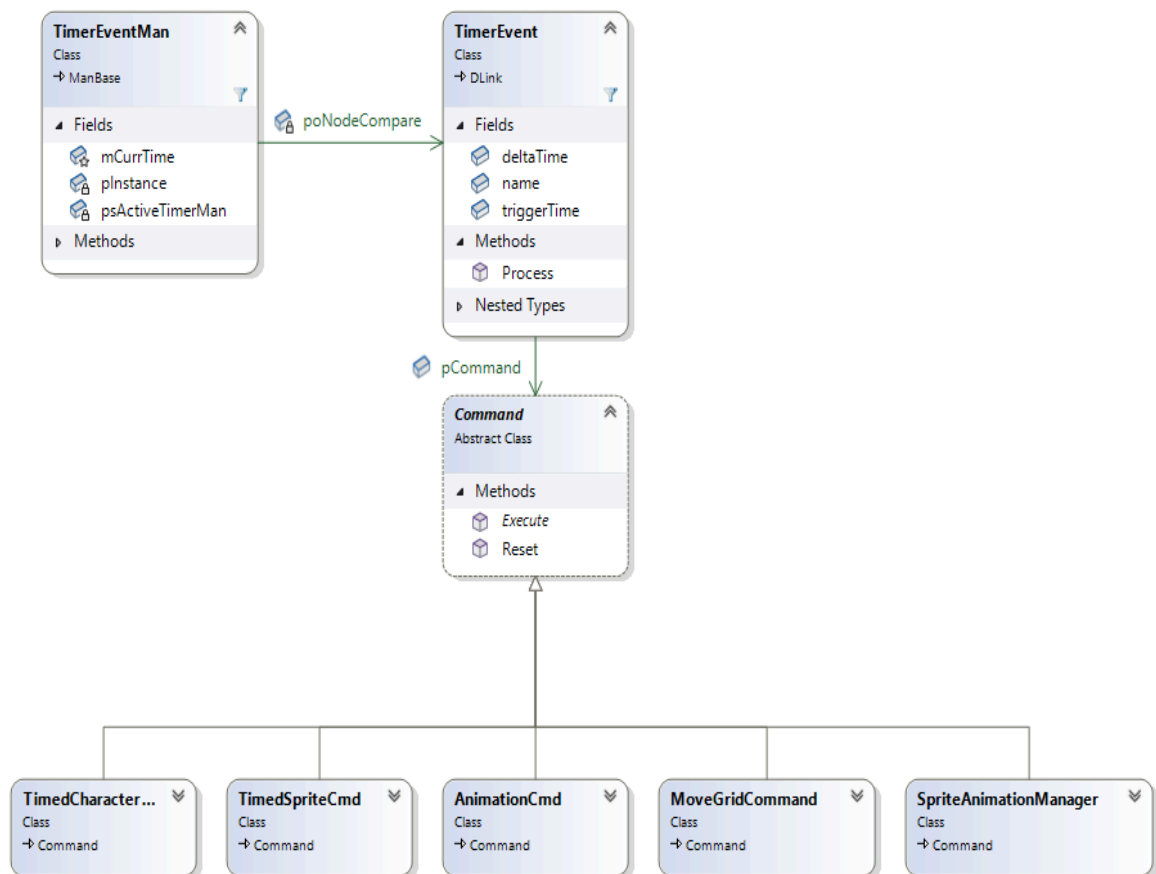
Command Pattern

Problem:

When developing a real-time application, integrating a timer event manager becomes crucial. This manager maintains a list of events scheduled for execution based on elapsed time. However, a challenge arises: how do we dynamically add events to the timer event manager and ensure they can repeatedly add themselves back into the timer? This is particularly relevant for Space Invaders, where various elements such as the alien march sound, grid movement, and animation rely on timer events. We require a mechanism for timer commands to autonomously repeat as necessary.

Solution:

The Command Pattern addresses the challenge of dynamically adding and managing timer events in real-time applications like Space Invaders by encapsulating updates within command objects. These objects, implementing a common interface, are added to the timer event manager, allowing for dynamic addition during runtime. Each command object executes its defined actions and can autonomously schedule itself for future updates if needed, promoting modularity and flexibility in managing game logic.



Pattern Description:

The command pattern transforms an action into an object, allowing you to store, queue, and parameterize these actions. Imagine wrapping a function call in a package - you can then pass this package around and execute it at any time, decoupling the object that initiates the action from the object that performs it. With the help of a Timer Event Manager, we can use the same command classes to re-add their action back according to the required time.

Key Object-Oriented Mechanics:

The Command Pattern employs several key object-oriented mechanics to achieve its functionality. At its core, it relies on abstraction through interfaces or abstract classes, defining a common interface for all commands. This abstraction allows commands to be treated uniformly, enabling polymorphic behavior. Inheritance is utilized to create concrete command classes that encapsulate specific actions or operations, ensuring flexibility and extensibility. These concrete classes override the execute method, implementing the specific behavior associated with each command.

Encapsulation is another crucial component since every command contains a particular action or activity, regardless of the person who initiates it. Encapsulation facilitates the addition, removal, and execution of commands without affecting the client code, hence promoting modularity and the separation of concerns. The pattern also uses composition to build sophisticated behaviors out of basic command objects. The ability for clients to combine commands into command queues, or macros, allows for more intricate control and choreography of actions inside the program.

Usage in Space Invaders:

In this project, the Command Pattern facilitates executing actions at specific times within the TimerEventManager. TimerEvent instances act as invokers, holding references to command objects to be executed. As the TimerEventManager processes timer events, commands are executed anonymously. The application utilizes movement, animation, and sound commands, which can be shared across various game objects and events. We use commands to help with repeated grid movement and animation in the play scene and to display timed text in the start and game over screens. Overall, the Command Pattern provides a straightforward solution for processing actions in the TimerEventManager without being tied to specific objects or events.

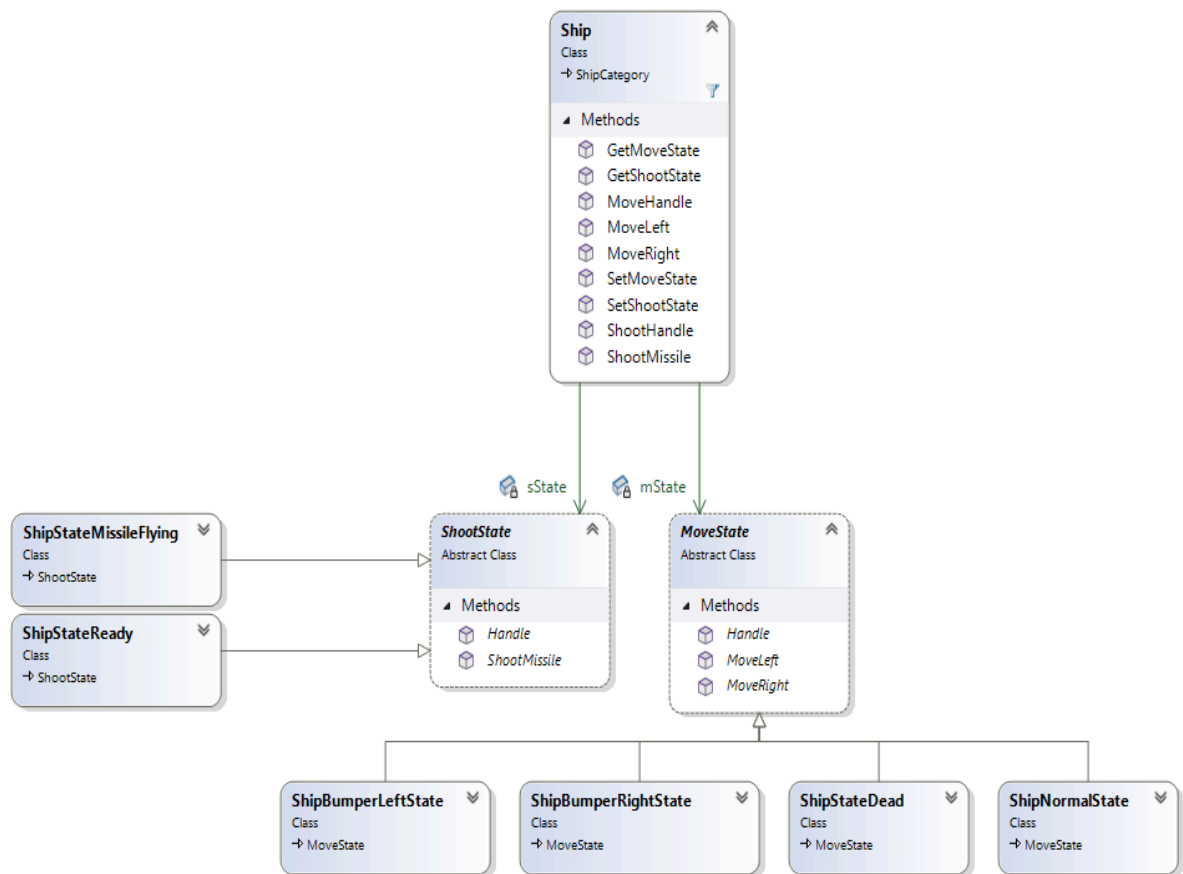
State Pattern

Problem:

Apart from merely registering input, it's crucial to impose constraints to prevent undesirable gameplay scenarios. For instance, the player should be unable to move the ship completely off-screen, limiting movement within defined left and right boundaries. Additionally, addressing the shooting mechanic presents a challenge; players must be restricted from firing another missile while one is already airborne. This limitation adds strategic depth, compelling players to exercise caution with their shots to avoid potential delays caused by missed attempts. We need a system that lets us limit the player's movement and actions based on various conditions.

Solution:

The state design pattern lets objects change their behavior based on their internal state. Think of it like a light switch - the switch itself (the object) stays the same, but its behavior (on or off) depends on its current state. To implement this, we need two things: a "context" object that represents the thing changing behavior, and a base class or interface for defining different states. Each state type implements this base class, providing the specific behavior for that state. This way, the context object seems to magically change its act depending on its internal state.



Pattern Description:

The state pattern is a behavioral design pattern that allows an object to alter its behavior when its internal state changes. It encapsulates state-specific behavior into separate classes and allows the object to switch between these classes dynamically. It is particularly useful when an object's behavior depends on its internal state and when the number of states is expected to change frequently.

Key Object-Oriented Mechanics:

The state design pattern's primary object-oriented principles are abstraction, polymorphism, and encapsulation. To achieve encapsulation, an object's behavior is divided into discrete state classes, enabling each state to contain overrides. Abstract state classes, or interfaces, define a common interface shared by all concrete state classes, and this is where polymorphism comes into play. This makes it possible for objects to communicate with states consistently, irrespective of how they are implemented. The key benefit of abstraction is that it promotes modular and adaptable design by enabling states to define behaviors regardless of the environment in which they are employed.

Moreover, the state pattern leverages inheritance to establish a hierarchy of state classes, with each concrete state class implementing its own unique behavior. This inheritance hierarchy enables code reuse and facilitates the addition of new states without modifying existing code. Additionally, the pattern often employs composition, where the context object maintains a reference to the current state, allowing it to delegate behavior to the current state object dynamically. This composition-based approach decouples the context from specific state implementations, promoting flexibility and extensibility in the system's design.

Usage in Space Invaders:

In Space Invaders, we use the state pattern to manage different behaviors for shooting and movement. This involved defining abstract states for shooting and moving, each with several concrete states. For shooting, it was straightforward—firing a missile changed the state to "missile flying," and upon collision, it returned to the "ready" state. This allowed me to limit the number of missiles a user can shoot at any given time to one on screen. In the case of movement, when the ship hits a left bumper, it enters a state preventing further left movement, and the same is true for the right bumper. The ship's default state was between these bumpers, but if the ship was destroyed, it lost control. Overall, the state pattern provided a solid framework for handling the ship's behavior transitions.

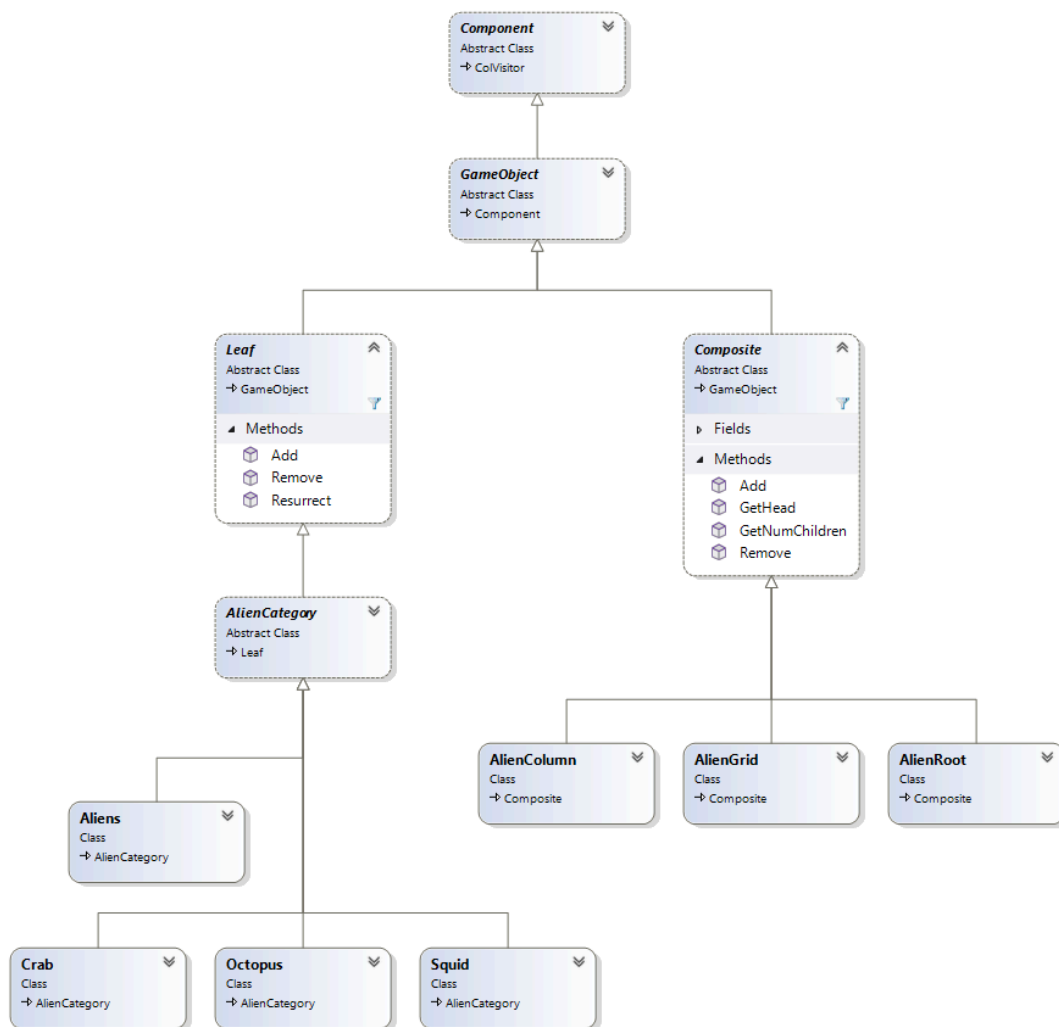
Composite Pattern

Problem:

In our game, we want the whole grid to move together. The challenge was to find an efficient way to move the grid as a whole rather than individually handling each alien sprite, especially when dealing with different types of aliens and dynamically changing grid sizes. It gets even trickier because we also need to track them in columns (for bomb drops).

Solution:

The composite pattern provides a solution by allowing us to treat individual objects and compositions of objects uniformly. In the context of space invaders, it enables us to organize aliens into grids and columns, creating a hierarchical structure where each grid or column can be treated as a single unit. This hierarchical organization facilitates the movement and management of aliens as a group, allowing us to handle grid-specific, column-specific, and alien-specific behaviors efficiently.



Pattern Description:

The composite pattern allows you to create tree structures from objects. Imagine building a family tree - each person (object) can have children (sub-objects). This lets you treat individual objects and groups of objects (composites) similarly. The key is that both objects and composites implement a common interface, allowing uniform operations regardless of whether you're dealing with a single object or a whole branch of the tree. This simplifies managing complex hierarchical structures.

Key Object-Oriented Mechanics:

The Composite Pattern enables developers to organize objects into hierarchical structures, where composite objects can contain leaf objects or even other composite objects. This hierarchical arrangement allows for uniform treatment of individual objects and composite objects, simplifying interactions for clients and facilitating dynamic changes to the structure at runtime. Despite its benefits, using the pattern introduces performance overhead due to traversing the composite structure, particularly evident in tasks like collision detection. Implementing the pattern also adds complexity to the creation of multiple classes and requires careful consideration of traversal strategies to ensure efficient tree traversal, often necessitating the use of a composite iterator class.

Usage in Space Invaders:

The composite design pattern provides a robust solution to managing the grid of aliens, allowing us to organize them into composites representing both columns and individual aliens. With this pattern, actions such as moving the aliens can be applied uniformly to the entire grid structure, regardless of whether it consists of columns or individual aliens. By leveraging the composite pattern, we can iterate through the components of the grid hierarchy and perform operations on each one, ensuring efficient and consistent behavior.

Moreover, the composite pattern allows for dynamic adjustments to the grid layout, making it easier to modify the quantity and configuration of aliens. This flexibility also applies to collision detection, where each component can have bounding boxes given to it. This simplifies collision tests inside grid segments rather than going through the full alien list. All things considered, the composite pattern makes managing hierarchical structures easier and provides a unified method for managing shared behaviors and interactions inside the application.

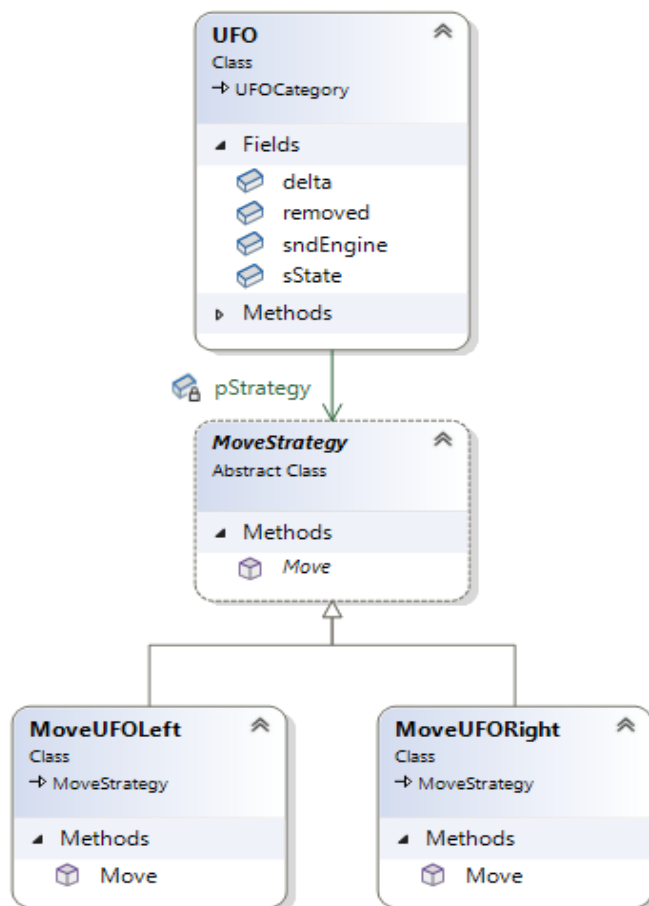
Strategy Pattern

Problem:

A challenge arises when attempting to create objects with similar appearances but different behaviors. Take, for instance, the UFOs in Space Invaders: while they share similar traits, those spawning from the left move right, and vice versa. This raises the question of how to differentiate behaviors among objects that outwardly appear the same. Similarly, various bomb types in the game exhibit slight differences in animation. This dilemma prompts the need to address how to imbue similar objects with distinct behaviors.

Solution:

The strategy pattern provides a solution by allowing different behaviors to be encapsulated within separate strategy objects. In the context of space invaders, each UFO or bomb type can have its own strategy object, representing its unique behavior. By dynamically assigning the appropriate strategy to each object, even those with similar appearances can exhibit distinct behaviors. For instance, UFOs spawning from different sides of the screen can be assigned strategies that dictate their movement direction accordingly. Similarly, bomb types can employ strategies tailored to their specific animation variations.



Pattern Description:

A behavioral design pattern called the strategy pattern makes it possible to define a family of algorithms, encapsulate them all, and make them interchangeable. This pattern offers a mechanism to choose the right algorithm at runtime by enabling the algorithms to change independently of the environment in which they are used. By essentially separating behavior from context, it fosters flexibility by enabling objects to dynamically alter their behavior by alternating between several approaches. This pattern comes in very handy when there are several iterations of an algorithm that must be dynamically chosen or alternated between while the program is running.

Key Object-Oriented Mechanics:

The strategy pattern leverages key object-oriented mechanics to achieve its flexibility and decoupling of behavior. Firstly, it employs abstraction through interfaces or abstract classes to define a common interface for all interchangeable algorithms, known as strategies. This abstraction allows different concrete strategies to implement the same interface, ensuring they can be used interchangeably within the context. Secondly, it utilizes polymorphism to enable objects to use different strategies interchangeably without altering their structure. By coding to the interface rather than the implementation, objects can dynamically switch between strategies at runtime, facilitating adaptability and reusability.

Usage in Space Invaders:

The UFO's spawning location and movement add complexity to its implementation. To address this, we employ the strategy pattern, integrating it into the UFO class. This involves incorporating a reference to a move strategy as a member variable within the UFO class, with two move strategies derived from the MoveStrategy abstract class: one for moving the UFO left and another for moving it right. By instantiating two UFO objects in the application, each with its own move strategy, we ensure flexibility and reusability.

With this setup, each UFO object can be tailored to behave differently based on its assigned move strategy. By parameterizing the UFO constructor with a MoveStrategy instance, we establish a clear association between each UFO instance and its specific movement behavior. As a result, UFOs can be spawned with distinct movement patterns, such as moving right on the left side of the screen and moving left on the right side, while maintaining the reusability and scalability of the UFO class.

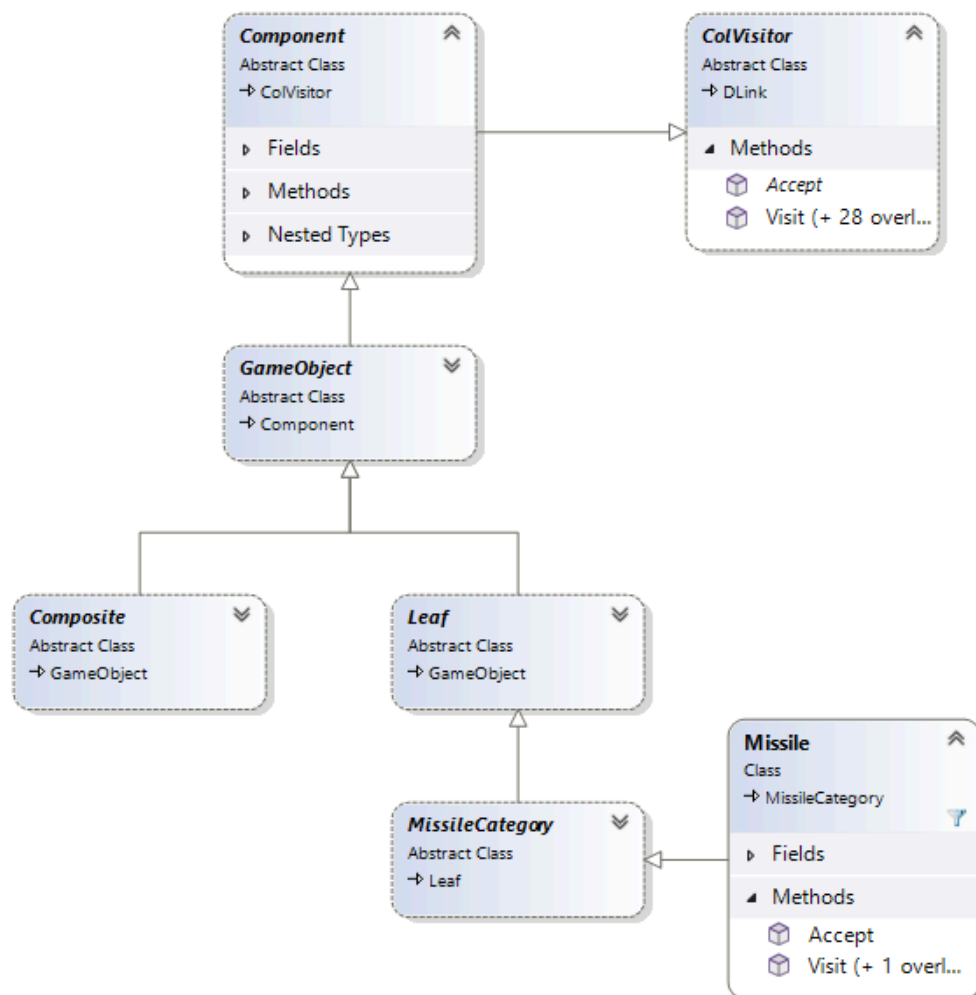
Visitor Pattern

Problem:

When dealing with collision detection in space invaders, the complexity escalates due to the game's diverse elements, such as multiple types of alien invaders nested in a collision hierarchy. Addressing questions like how collisions are detected and which objects collide becomes crucial. To manage this complexity effectively, a collision detection and resolution system must be designed to handle collisions between projectiles, invaders, the player's ship, and the wall system, considering the diverse behaviors and states of these game elements.

Solution:

The visitor design pattern simplifies collision management for space invaders by separating collision detection logic into specialized visitor classes. These visitors handle specific collision scenarios between game elements. By decoupling collision detection from the game element classes, the pattern enables a modular and extensible approach to handling collisions.



Pattern Description:

A behavioral design pattern called the Visitor pattern enables the addition of new behaviors to pre-existing classes without changing their organizational structure. This is accomplished by keeping the algorithm independent of the object structure it works with. Visitors in the pattern stand in for actions that need to be taken on various object structure aspects. Elements accept visitors, which then traverse the structure and perform operations specific to each element. This technique is especially helpful in circumstances where classes remain stable but behaviors need to change.

Key Object-Oriented Mechanics:

The Visitor pattern provides developers with a modular approach to solving various types of problems, particularly in collision processing scenarios. It offers flexibility by allowing visitors or other elements to be added as needed. Treating the visitor as an abstract class rather than an interface simplifies implementation, as it enables the implementation of only the necessary visit methods as virtual methods. This approach allows derived classes to focus on implementing visit methods specific to the objects they interact with, promoting a cleaner design and reducing unnecessary overhead.

This separation facilitates the addition of new visitor operations without requiring modifications to existing classes unless necessary. Additionally, creating new concrete visitor classes becomes straightforward, as they are not obligated to implement a visit method for every possible kind of element.

Usage in Space Invaders:

The visitor pattern efficiently addresses our collision problem by defining visit methods as virtual in the abstract visitor class. This approach allows us to implement collision handling only for object pairs where collisions are possible, offering flexibility and reducing unnecessary overhead. Notably, in our project, we didn't employ a separate element base class. Instead, both visitor and element functionalities are incorporated within the same class, streamlining the system. This setup enables all visitors to also serve as elements, facilitating seamless interaction within the collision system. Each game object inherits from the visitor class, granting them the capability to implement collision systems if required, although it's not mandatory.

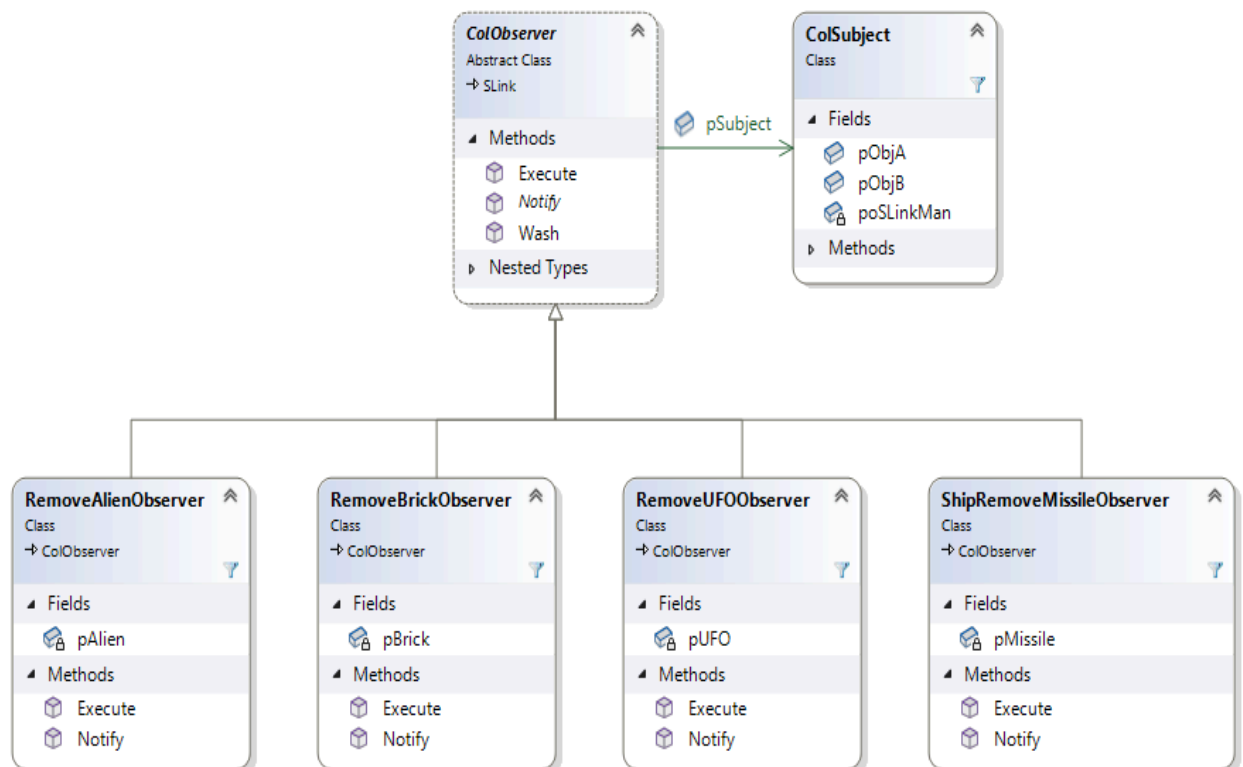
Observer Pattern

Problem:

It is important that collisions cause suitable reactions or actions from all concerned parties. Collisions are occurrences that are supposed to cause certain things to happen, including removing a shield when a missile or bomb hits it, making objects vanish when they collide, or altering a ship's status when it hits a bumper. To ensure consistent behavior, the problem is to come up with a way to identify collisions and connect them to relevant actions or events.

Solution:

In the context of collision detection, the observer pattern allows collision events to be detected independently from the objects involved. Each object registers itself as an observer interested in certain collision events. When a collision occurs, the collision detection system notifies all registered observers, triggering the appropriate response for each object based on its registered interests.



Pattern Description:

An object, referred to as the subject, keeps track of its dependents, referred to as observers, and notifies them of any state changes, typically by invoking one of their methods. This is known as the observer pattern, a behavioral design pattern. One-to-many relationships between items are made possible by this design, which guarantees that modifications to the state of one object can effectively propagate to several other objects without explicitly linking them. It is possible for observers to dynamically attach to and detach from the subject, which enables adaptable and loosely coupled interactions between system components.

Key Object-Oriented Mechanics:

The observer pattern revolves around two fundamental elements: the subject and the observer. The subject encapsulates the state being observed and manages a list of dependent observers. It provides methods for attaching new observers, detaching existing ones, and notifying them of any changes to their state. On the other hand, observers are entities interested in the state changes of the subject. They register themselves with the subject and define an update method to be invoked whenever the subject's state changes. This mechanism enables a flexible and decoupled interaction between the subject and observers, allowing multiple observers to react dynamically to changes in the subject's state without the need for direct dependencies between them.

Usage in Space Invaders:

The observer pattern provides a straightforward solution to triggering events upon collisions. By establishing a collision subject that holds the objects involved in a collision pair and notifying registered observers, the pattern enables dynamic responses to collision events. For instance, to handle the destruction of an alien and a missile upon collision, observers are attached to the collision subject. As the collision occurs, the subject notifies the registered observers, prompting the execution of specific actions. Different observers handle different tasks; we have set up observers to remove shields, bricks, aliens, and UFOs. This shared convention simplifies the notification process, ensuring consistent behavior across different collision scenarios.

Similarly, collisions and state switches for the ship and UFO are triggered with the help of observers. For each scenario, specific observers are attached to the collision subject, allowing for tailored responses.